Generalized Linear Models

Today's agenda:

- Assignment 4
- GLMs (9.3 - 9.4)

Assignment 4

Similar to assignment 4, but using lm, glm, etc. (i.e., Chapter 9)

- Note: can convert models from previous assignments into the lm()/glm(), but you should also test a few new models as part of this exercise.

Assignment 4

Identify one or more biologically interesting questions, then:

- Construct several candidate models (for each question)
- For each model, you should:
  - provide documentation as to why you constructed the model as you did
  - Explain the biological/ecological interpretation of that model
- Identify which model(s) you think are best (for each question)

Assignment 4

Full details are laid out on the assignment page (linked from Canvas)

Turn in as usual (file or Github link on canvas)

Assignment 4

Questions?

Previously

We talked about lms and fitted a bunch

- These all focus on linear models and normal errors
- Today we cover models that are a bit more flexible

Nonlinear Least Squares

- Don’t require linearity
- Still requires a normal distribution
- Very similar syntax to lm()

Nonlinear Least Squares

Load packages

```
library(readr)
library(bbmle)
library(tidyverse)
library(emdbook)
```

Get tadpole data:

```
data(ReedfrogFuncresp)

plot(ReedfrogFuncresp$Killed ~ ReedfrogFuncresp$Initial)
```

Nonlinear Least Squares

Dataset has one predictor (Initial #) and one response (# killed)

- But there could be some non-linearity there?

To fit a lm we would use:

```
frog.lm <- lm(Initial ~ Killed,
              data = ReedfrogFuncresp)
```

Run this now

Nonlinear Least Squares

```
frog.lm <- lm(Initial ~ Killed,
              data = ReedfrogFuncresp)
```

The equivalent nls call is:

```
frog.nls.0 <- nls(Initial ~ int + a*Killed,
                  data = ReedfrogFuncresp)
```

What is different in the model specifications?

Run the code for the nls and compare AICs

Nonlinear Least Squares

So far the models aren't non-linear.
Let's allow nonlinearity:

```
frog.nls.1 <- nls(Initial ~ int+ a*Killed^b,
                  data = ReedfrogFuncresp)
```

Run this and compare the AICs with the previous models.
Which is best?

Nonlinear Least Squares: fir trees

Let’s try a different example:

```
data(FirDBHFec_sum)
FirDBHFec_sum <- na.omit(FirDBHFec_sum)
 plot(FirDBHFec_sum$fecundity ~ FirDBHFec_sum$DBH)
```

This dataset has one response (fecundity), two predictors (DBH, pop)

Appears to be stronger non-linearity here

Nonlinear Least Squares: fir trees

Write an nls() model equivalent to:

```
lm(fecundity ~ DBH, data=FirDBHFec_sum)
```

Nonlinear Least Squares: fir trees

You should have something like:

```
fir.nls.0 <- nls(fecundity ~ int+ a*DBH,
                 data = FirDBHFec_sum)
```

How might we add in non-linearity?

Make a model that allows non-linearity and compare the AICs. Which is better?

nls with categorical data

What if we want to add in a categorical variable?

- Use Generalize Nonlinear Least Squares (GNLS)
  - Syntax similar to mle2
- Use nlsList()
  - Fits separate models for each level

nlsList

```
library(nlme)
fir.nlslist <- nlsList(model = fecundity ~ int + a*DBH^b | pop,
                       data = FirDBHFec_sum,
                       start = coef(fir.nls.1))

summary(fir.nlslist)
```

- The “| pop” tells it to fit this model for each category in pop
- Since it fits multiple models, it gives multiple AIC values
  - `AICtab(fir.nlslist, base = TRUE)`

gnls

Here's our best nls model:

```
fir.nls.1 <- nls(fecundity ~ int+ a*DBH^b,
                 data = FirDBHFec_sum)
```

Here's how the gnls version looks:

```
fir.gnls.0 <- gnls(fecundity ~ int + a*DBH^b,
                   data = FirDBHFec_sum,
                   params = list(int ~ 1, a ~ 1, b ~ 1),
                   start= list(int = 0.58, a = 0.5, b = 2),
                   control = list(tolerance = 10))
```

Run this model now

gnls

```
fir.gnls.0 <- gnls(fecundity ~ int + a*DBH^b,
                   data = FirDBHFec_sum,
                   params = list(int ~ 1, a ~ 1, b ~ 1),
                   start= list(int = 0.58, a = 0.5, b = 2),
                   control = list(tolerance = 10))
```

Params: ~ 1 specifies that there aren't group-specific values

Start: start values needed for each parameter

Control: not always needed, but here helps fitting

gnls

Let's modify this to allow for population-specific exponents

```
fir.gnls.1 <- gnls(fecundity ~ int+a*DBH^b,
                   data = FirDBHFec_sum %>% na.omit(),
                   params = list(int ~ 1, a ~ 1, b ~ pop),
                   start= list(int = 0.58, a = 0.5, b = c(2,2)),
                   control=list(tolerance=10))
```

- Try this and see how it compares to the other models.
- Then, make a model that has the intercept varying by population
  - How does that compare?

GLMs

What if we have a non-normal distribution?

That's where Generali**zed** Linear Models come in

Allows for non-normality

- Distributions must be in the exponential family
  - Includes: normal, poisson, binomial, gamma and others
- See `?family` for options

Syntax is similar to lm() but you specify the family (i.e., the distribution to use)

GLMs

We'll grab some data on publishing trends in ecology and evolutionary biology

- Includes binary responses (code/data shared, Binomial)
- Includes citation rate (0 to Inf, Right skewed, Poisson is sensible)

```
code_data <- read_rds("https://tinyurl.com/codesharingdata")
```

Run this and load the data

GLMs: Binomial

Here's how we'd write the lm:

```
code.lm  <- lm(formula = r_scripts_available ~ year + data_available +
                         open_access,
               data = code_data)
```

Here's how we write the glm:

```
code.glm1  <- glm(formula = r_scripts_available ~ year + data_available +
                            open_access,
                  family = binomial,
                  data = code_data)
```

Run both of these and compare AICs

Predict()

The AIC isn’t the only indicator to use.
Let’s look at the quality of the model predictions using predict()

predict() allows you to get predictions with a fitted model object

- By default gives predictions for the original data
- Can also predict using NEW data

Predict()

```
plot(predict(code.lm) ~ code_data$year)
abline(h = 0)

plot(plogis(predict(code.glm)) ~ code_data$year)
abline(h = 0)
```

Note: plogis() is needed to transform the predictions back to the original scale.

- Since GLMs use linearizing functions, you need to back-transform predictions

Run these now

Predict()

We can also use predict to get confidence and prediction intervals:

```
predict(code.lm, interval = "confidence")

predict(code.lm, interval = "prediction")
```

Notice anything about the predicted values?

Predict()

- Data clearly binary
- Normal model makes impossible predictions
- Support choice of binomial, regardless of AIC

What about AIC?

- It doesn’t work when comparing continuous vs discrete
- This is a cautionary tale!
- Fine to compare continuous vs continuous, or discrete vs discrete

Poisson

```
citations.glm0 <- glm(data = code_data,
                      family = "poisson",
                      formula = citations ~ age_y)
```

Run this model

Does age explain the number of citations?

Challenge

- Try to improve `citations.glm0`
- Lots of other variables in the dataset you can try adding as predictors
- Ask if you don’t know what a field contains

Before next class:

- Read “R for Data Science” chapters 3 and 4:
  - https://r4ds.hadley.nz/data-transform.html
  - https://r4ds.hadley.nz/workflow-style.html
- May be a quiz on these